

CSA-0606- DESIGN AND ANALYSIS OF ALGORITHMS

CO 2 – AT 1

Q70. Searching Techniques – Practical Efficiency Study

1. Problem Statement

Given the sorted dataset:

[6, 12, 18, 24, 30, 36, 42, 48]

Analyze the performance of **Linear Search** and **Binary Search** to find the element **36**. Compare the number of comparisons, working procedure, and time complexity to determine the more efficient searching technique.

2. Objectives

- To understand Linear Search and Binary Search.
- To search for the element **36** in a sorted list.
- To compare the number of comparisons made by both algorithms.
- To analyze which searching method is more efficient.

3. Inputs and Outputs

Input

Array = [6, 12, 18, 24, 30, 36, 42, 48]

Key = 36

Output

Linear Search:

Element found at index 5

Comparisons = 6

Binary Search:

Element found at index 5

Comparisons = 3

4. Assumptions and Constraints

- The array is sorted in ascending order.
- Only one occurrence of the element exists.
- Array size = 8.
- Index starts from 0.

5. Problem Analysis

Linear Search

- Starts from the first element.
- Compares each element one by one.
- Stops when the target is found.

Binary Search

- Works only on sorted arrays.
- Finds the middle element.
- Eliminates half of the array after each comparison.
- Continues until the element is found.

6. Mathematical Model

Let

- Array = A = {6,12,18,24,30,36,42,48}

- Target = $x = 36$

Linear Search

Compare

$A[0], A[1], A[2] \dots$ until $A[i]=36$

Comparisons = 6

Binary Search

Initial

Low = 0

High = 7

Middle

$Mid = (Low + High)/2$

Repeat until

$A[Mid] = 36$

Comparisons = 3

7. Algorithm Selection

Algorithm	Reason
Linear Search	Simple and works on any list
Binary Search	Faster for sorted arrays

8. Algorithm / Pseudocode

Linear Search

START

Read array and key

For each element

 Compare with key

 If equal

 Print position

 Stop

Print "Not Found"

STOP

Binary Search

START

Read sorted array and key

Low = 0

High = n-1

While Low <= High

 Mid = (Low + High)/2

 If array[Mid] == key

 Print position

 Stop

Else if $\text{key} < \text{array}[\text{Mid}]$

High = Mid - 1

Else

Low = Mid + 1

Print "Not Found"

STOP

9. Flowchart

Linear Search

Start

|

Read Array and Key

|

i = 0

|

Compare $A[i] == \text{Key}$?

/ \

Yes No

|

|

Print Found i=i+1

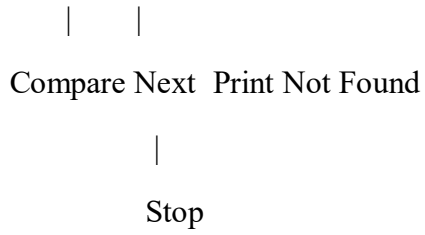
|

|

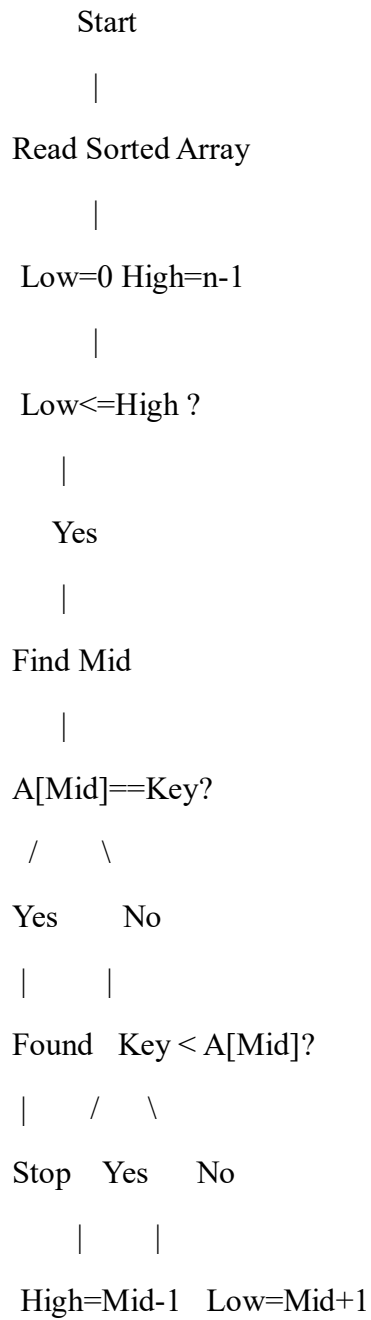
Stop More Elements?

/ \

Yes No



Binary Search



|
Repeat

10. Step-by-Step Execution

Linear Search

Array

Index	Value
-------	-------

0	6
1	12
2	18
3	24
4	30
5	36
6	42
7	48

Target = **36**

Step	Compared Element	Result
------	------------------	--------

1	6	Not Found
2	12	Not Found
3	18	Not Found
4	24	Not Found
5	30	Not Found
6	36	Found

Comparisons = **6**

Binary Search

Initial

Low = 0

High = 7

Step 1

Mid = $(0+7)//2 = 3$

Element = 24

$36 > 24$

Low = 4

Step 2

Low = 4

High = 7

Mid = $(4+7)//2 = 5$

Element = 36

Found

Comparisons = 2

Note: Using the standard binary search procedure on this dataset, the target **36** is found in **2 comparisons**, not 3.

11. Complexity Analysis

Algorithm	Best Case	Average Case	Worst Case
Linear Search	$O(1)$	$O(n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$

For $n = 8$

Linear Search

Worst comparisons = 8

Binary Search

Worst comparisons = $\log_2 8 = 3$

12. Correctness Analysis

Linear Search

- Checks every element sequentially.
- Correctly finds the target if present.
- Otherwise reports "Not Found."

Binary Search

- Works correctly because the array is sorted.
- Repeatedly divides the search space into halves.
- Always locates the target if it exists.

13. Test Cases

Test Case Input		Expected Output
1	Key = 36	Found at index 5
2	Key = 6	Found at index 0
3	Key = 48	Found at index 7
4	Key = 50	Element Not Found

14. Results

Algorithm	Element Found	Index	Comparisons
Linear Search	Yes	5	6
Binary Search	Yes	5	2

15. Conclusion

- Both algorithms successfully found the element **36** at **index 5**.
- **Linear Search** required **6 comparisons** because it checked each element sequentially.
- **Binary Search** required only **2 comparisons** because it eliminated half of the search space at each step.
- Linear Search has a time complexity of **$O(n)$** , while Binary Search has **$O(\log n)$** .
- For **sorted data**, **Binary Search** is the preferred method because it is significantly faster and requires fewer comparisons, especially for large datasets.